

Case Study: Online Art Gallery DBMS

Introduction

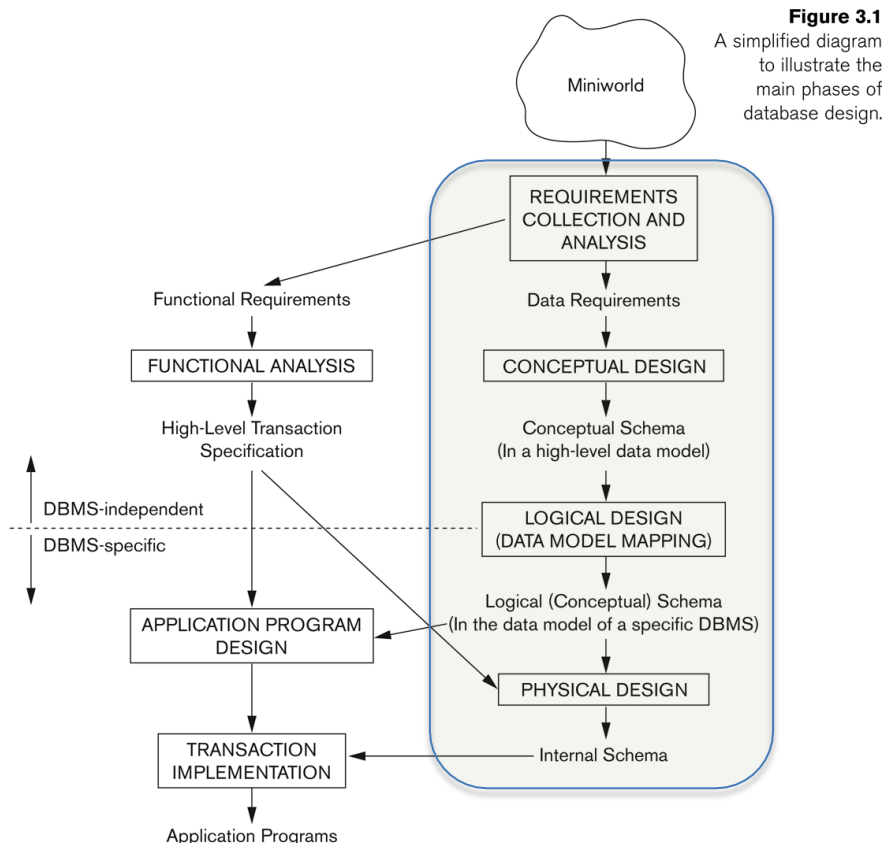
This case study presents the design and implementation of an Online Art Gallery Database Management System (DBMS). The system allows artists to register and upload their artworks for display, while customers can browse, purchase, and confirm receipt of artwork. The payment is handled externally through Zelle, and shipping is managed by the artist. Once the customer receives the artwork, the order is archived.

Objectives

- Provide a platform for artists to showcase their work
- Enable customers to browse, purchase, and receive artworks
- Facilitate the confirmation and tracking of orders
- Archive orders upon customer receipt confirmation

Navathe: slide 3-5

Overview of Database Design Process- In this course, we will focus on the shaded portion.



Step 0: Requirement Analysis (studied in Software Engineering)

Actors: Who will use the system?

Artist

Customer

Admin

Use cases: What functionalities do they want in the system?

Artist: register, upload art works, monitor sales

Customer: register, order, monitor orders

You must have studied requirement engineering in software engineering course. We will focus on Database Design and Implementation.

Step 1: Identify Input and Output Requirements (check step 5 also)

Inputs:

- Artist registration details (first name, last name, email, Bio, ...)
- Artist upload Artwork details including image (title, description, date, image,...)
- Customer registration (first name, last name, email, ...)
- Customer Order and payment information (artwork, pay-method, referNo,...) -notify Artist
- Artist: payment received confirmation and shipping information (shipMethod, trackingNo, ..) -notify Customer
- Order Received- confirmation by customer (notify Artist and Admin)

Outputs:

- Artworks Gallery (list of artworks (image, title, artist, price range)
- Admin, Artist, and Customer dashboards
- Customer order history (list of orders-completed and current (status)
- Artist sale history
- Order status view- status history
- Archived order logs

Step 2: Conceptual Design

Lucid Chart: <https://youtu.be/wR0ig0eQsZA>

Lucid Chart: <https://youtu.be/QpdhBUYk7Kk>

<https://www.youtube.com/watch?v=9L0CMnDGuLg&t=656s>

Study Ch-3 chapter & slides (& Ch- 4 -Navathe)

Determine Entities, and their key attributes, and the relationships. Also identify the participatory and multiplicity constraints (cardinality).

Identify Entities: An **entity** is a thing you need to store information about. It can be:

- A person (e.g. Student, Employee, Customer)
- A place (e.g. Office, Branch)
- An object (e.g. Product, Book, Vehicle)
- An event (e.g. Order, Sale, Appointment)
- A concept (e.g. Course, Department, Account)

Read the problem description, and check words---

Is it a noun? Maybe Entity

Does it have multiple attributes? Likely entity

Can it exist on its own? Entity

Do we need to track multiple instances? Entity

Is it a relationship? Not entity, but association

Case Study: Key Entities: Artist, Artwork (weak), Customer, Order

Note: Artwork belongs to a particular Artist, but it may not be weak if we like to give unique identification number independent of its owner.

Key Attributes:

- Artist: ArtistID, Name, Email, PasswordHash, Bio, ProfilePic
- Customer: CustomerID, Name, Email, PasswordHash, Address
- Artwork: ArtworkID, ArtistID, Title, Description, Price, ImageURL, Status
- Order: OrderID, OrderDate, OrderStatus, paidDate, payConfirmNumber,,paymentReceivedDate, shippedDate, shippingCarrier, trackingNumber,, receivedDate, archived)
- Admin: adminID, firstName, lastName, email, username, passwordhash

Relationships & Cardinality

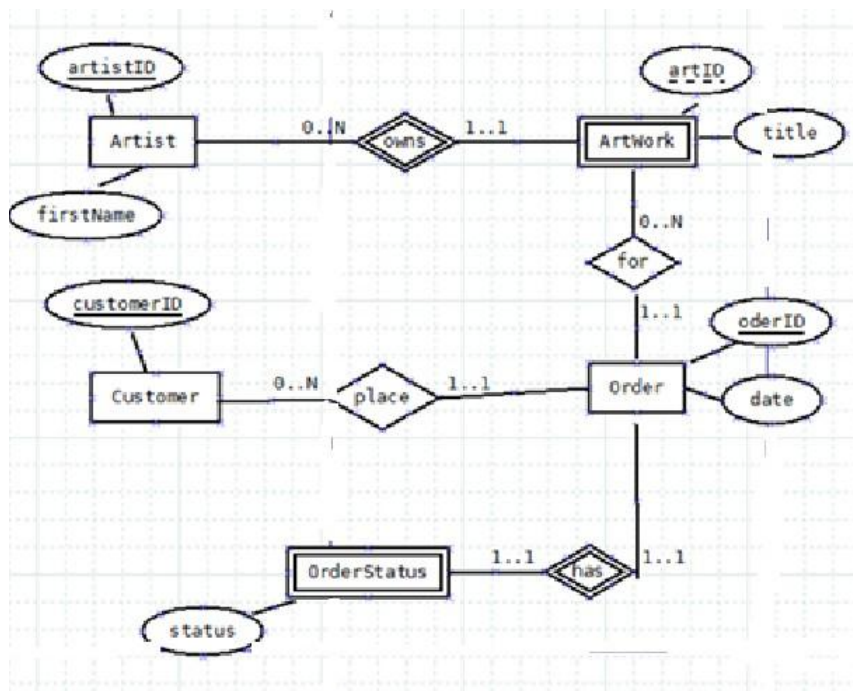
- One Artist can upload zero (none yet), one or many Artworks – there should be an artist linked with an artwork.

- One Customer can place zero (none now), one or many Orders - there should be a customer linked with an order.
- One Order is linked to exactly one Artwork - there may be zero (none yet), one or many orders on an artwork
- One Order is linked to exactly one OrderStatus and vice versa (may be combined)

Inclusion of OrderStatus as a separate table supports tracking lifecycle stages of orders, which is scalable and extensible.

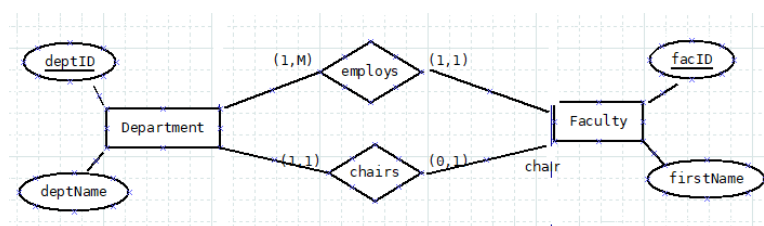
Draw the ER Diagram

The following diagram is drawn using Dia.

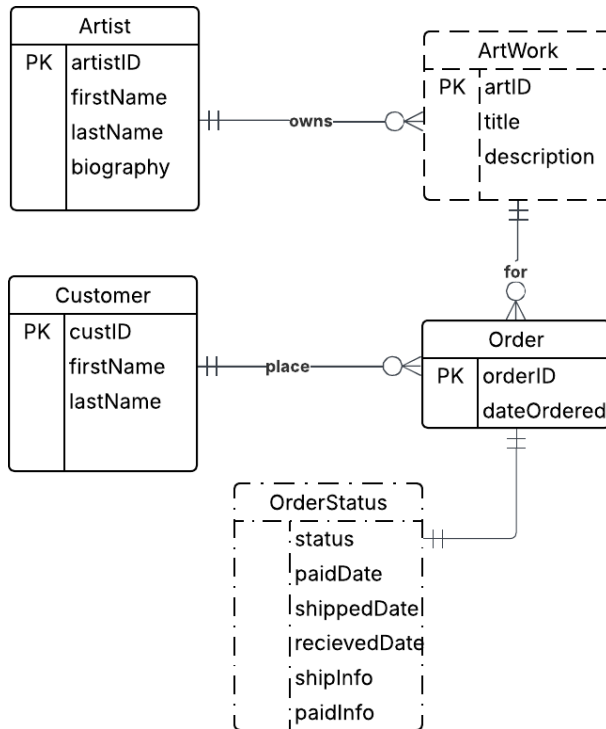


Reading ER diagram symbols: e.g. Artist owns (0 or many) art works

Example: Slide: Nava 3-44: Read the (min,max) numbers next to the entity type and looking **away from** the entity type. For example, read the following as- A Department employs (1 or more) Faculty. A faculty member is employed by (1 and only 1) department.



Lucid chart is used to create the diagram below. This notation is generally used for next level design (logical). You can later export this diagram to tables (SQL code). More attributes will be included later in logical design step.

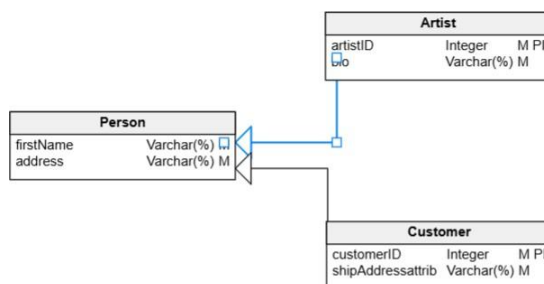


E-ER Diagram

Slide 3-24: **Generalization Abstraction:**

Artist and Customer have many common properties (attributes, actions, relationships, etc).

This may be overlapping- An artist may be a customer also.



Note: EER-D notation is slightly different

Step 3: Logical Design

Study ch-4 slides:

Mapping ER Diagrams to Relational Schemas

- Relations are represented physically as **tables**
- Tables are related to one another
- Table holds information about entities (objects)
- **Rows** (tuples) correspond to individual records
- **Columns** correspond to attributes
- A column contains values from one domain
- Domains consist of atomic values
- Relations never have duplicate tuples, can always tell tuples apart; implies there is always a key (worst case – composite of all attributes)
- Superkey: set of attributes that uniquely identifies tuples
- Candidate key: superkey such that no proper subset of itself is also a superkey (i.e., it has no unnecessary attributes)
- Primary key: candidate key chosen for unique identification of tuples
- Cannot verify a key by looking at an instance
- Instance can show an attribute, or combination is not a key, due to duplicates
- Need to consider semantic information to ensure uniqueness – show duplicates are not possible in the mini-world
- A foreign key is an attribute or combination of attributes that is the primary key of some relation (called its home relation)

Designing Tables: (slide 3-31)

- Each strong entity set become a table
 - Non-composite, single-valued attributes become attributes of table
 - Composite attributes: either make the composite a single attribute or use individual attributes for components, ignoring the composite
 - Multi-valued attributes: remove them to a new table along with the primary key of the original table; also keep key in original table
- Weak entity sets become tables—add primary key of owner entity
- Binary Relationships:
 - 1:M – place primary key of 1 side in table of M side as foreign key

- 1:1 – use either key as foreign key in the other table
- M:M – create a relationship table with primary keys of related entities, along with any relationship attributes
- Ternary or higher degree relationships: construct relationship table of keys, along with any relationship attributes
- Recursive relationship – use foreign key if 1:M; use relationship table if M:M

Entities → Tables

Create a table for each entity with available attributes. Consider weak entities separately (see below). Later we will include more attributes.

Customer

customerID	firstName	lastName	eMail		
240111111	Ray				
240111112	Liz				

Artist

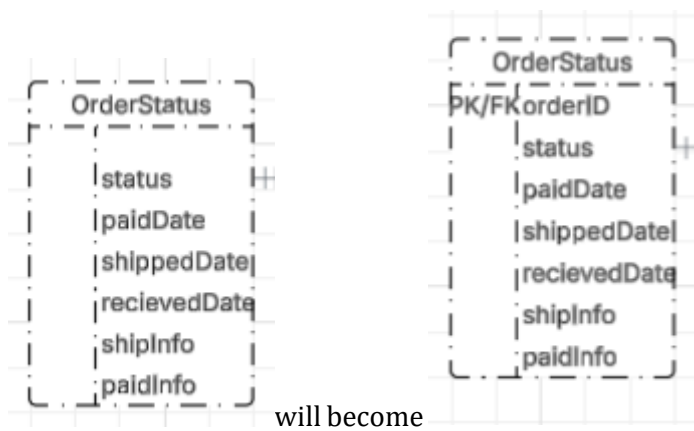
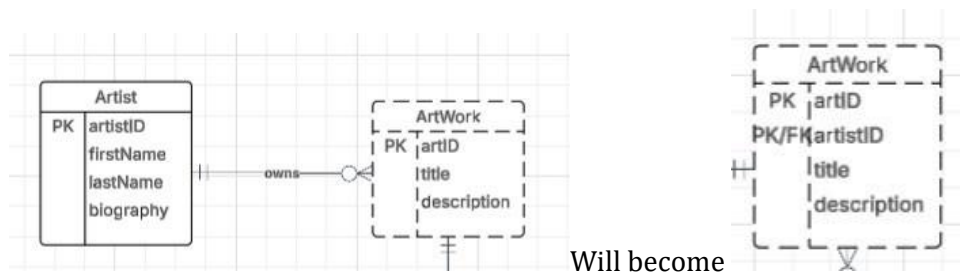
artistID	firstName	lastName	bio	eMail	
1000123	Sam				
1000124	Mary				

Order

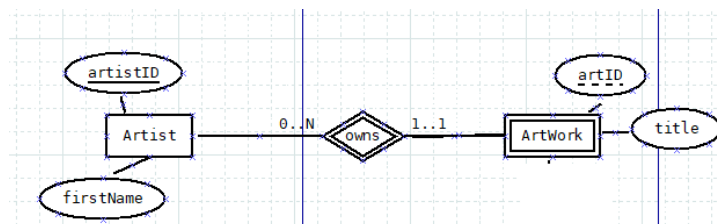
orderID				dateOrdered	
1111111					
1111112					

Weak Entities → Tables and add FK of the owner entity:

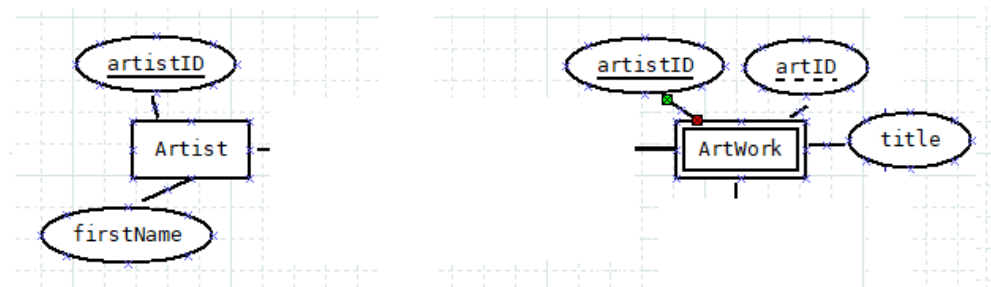
The owners Primary Key (PK) will be included in the weak entity as a Foreign Key. Both owners primary key and the weak entities weak key will be the primary key for the weak entity.



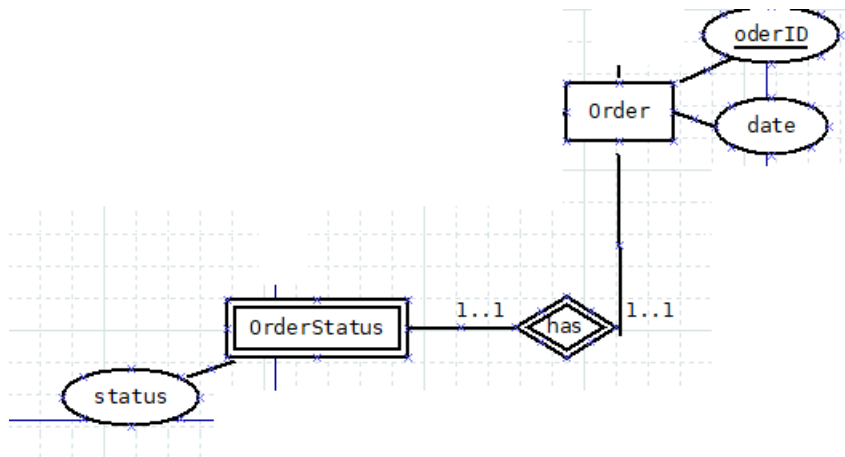
OR



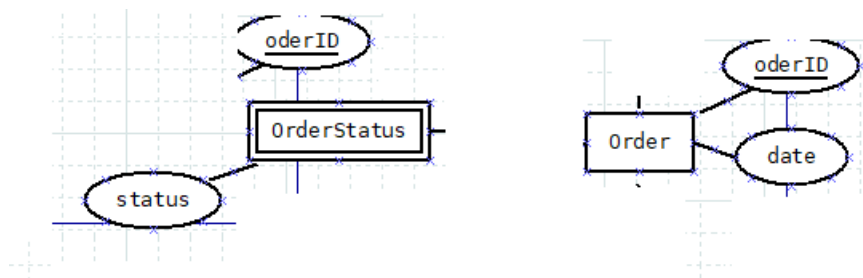
Will become



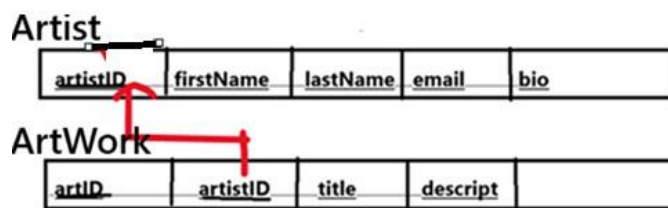
and

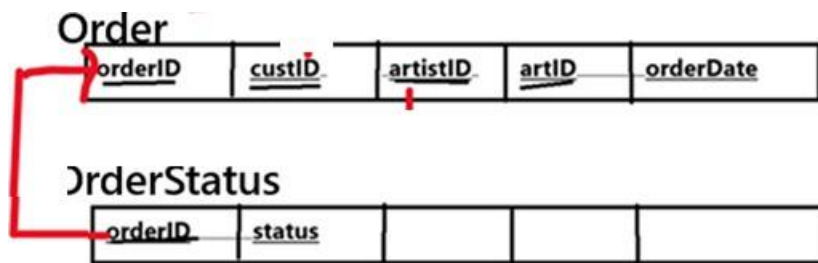


Will become



Since an Artist may have more than one Artwork (1-N), we need a weak key for Artwork (say, 1, 2, etc..). But, there is no need for a weak key for orderstatus (1 -1)





Mockup Tables

Artist

artistID	firstName		bio		
1000123	Sam				
1000124	Mary				

ArtWork

artistID	artWorkID	title	description	date	price
1000123	001				
1000124	003				

Order

orderID				dateOrdered	
1111111					
1111112					

OrderStatus

orderID	status				
1111111	paid				
1111112	shipped				

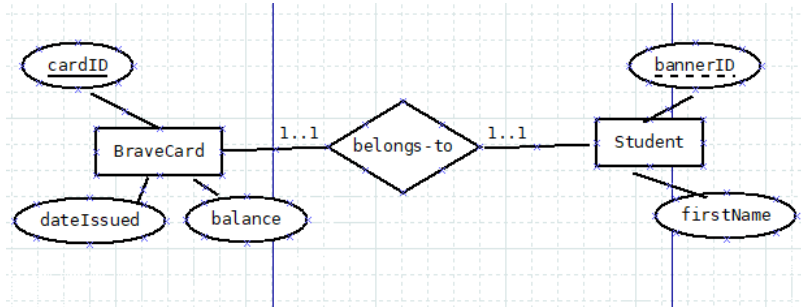
Designing Relationships.

One-One Relationships:

No (1-1) relationship in this case study. Therefore, we will use an example.

Example: Students are issued Brave cards. Every student should have exactly one brave card and one card belongs to exactly one student.

Conceptual Design



Banner ID is included in the BraveCard entity as an FK. When a card is swapped the student can be identified. Need not to include the card id in the student (to check the balance in the card)

Relationship is modelled as linking FK → PK

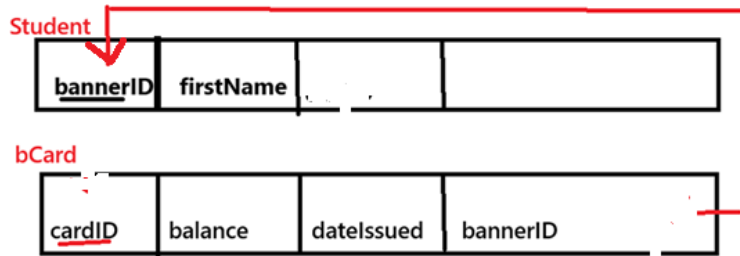
Logical Design

- Student (bannerID, firstName,)
- Card (cardID, bannerID, balance,...)
 - balanceID is included as FK- links to Student.bannerID

OR



OR



Mockup Tables: (make sure ids match)

Student

bannerID	firstName				
840111111	Ray				
840111112	Liz				

BraveCard

bCardID	balance	dateIssued			bannerID
123123123	1000				840111112
123123124	900				840111111

Another Example:

For 1..1 relation, place the FK only on one side. Either side is okay if both sides has same cardinality. Otherwise, place the foreign key in the table of the entity that has total participation, or the one that cannot exist without the other (to avoid null values in FK)

Example: Person and Passport

- One person has exactly one passport
- One passport is assigned to exactly one person

But suppose:

- Each passport must belong to a person (total participation for Passport)
- A person may not yet have a passport (partial participation for Person)

Also most likely query would be, who is the owner of given passport (passportID), rather, what is the passportID of the given person who is the

Then, Passport table should include PersonID as a FK.

In the next step we will create SQL code to create those table.

Table: Person

```
Person (  
    PersonID INT PRIMARY KEY,  
    Name VARCHAR(100)  
);
```

Table: Passport

```
Passport (  
    PassportID INT PRIMARY KEY,  
    PersonID INT UNIQUE,  
    FOREIGN KEY (PersonID) REFERENCES Person(PersonID)  
);
```

Can refer both ways. The table Passport is indexed on both personid and passportid automatically

1. Find Passport ID Given Person Name

```
SELECT p.Name, ps.PassportID  
FROM Person p  
JOIN Passport ps ON p.PersonID = ps.PersonID  
WHERE p.Name = 'John Doe';
```

2. Find Person Name Given Passport ID

```
SELECT ps.PassportID, p.Name  
FROM Passport ps  
JOIN Person p ON ps.PersonID = p.PersonID  
WHERE ps.PassportID = 123456;
```

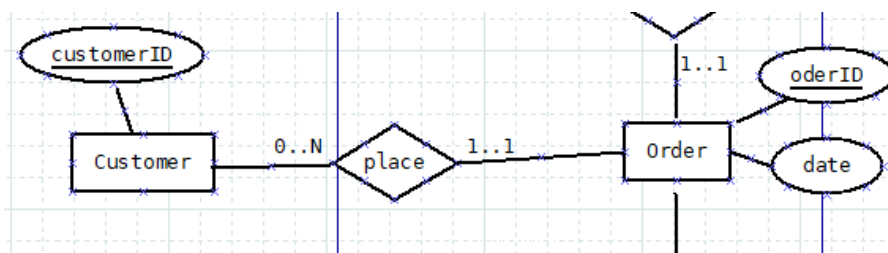
One-Many Relationships

The PK of the Many part should be included in the One part.

N this case study, there are two 1-N relationships with Order entity.

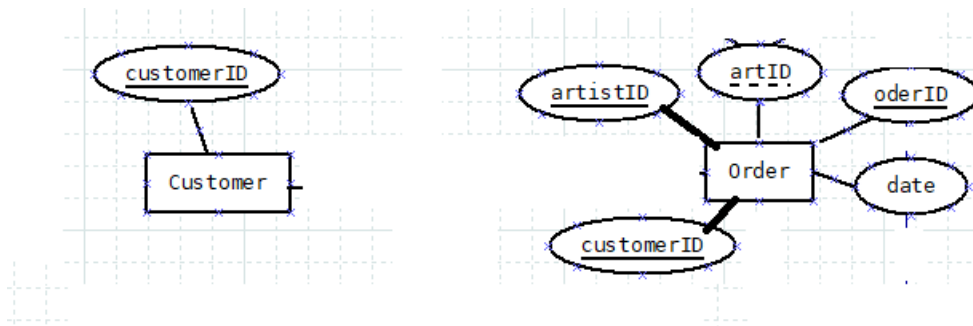
First consider Customer and Order relationship.

There are two 1-N relationships with Order entity.



Include PK of customer entity, customerID in the Order entity.

Similarly include PK of ArtWork entity, artworkID and artistID in the order entity.



Customer

<u>custID</u>	firstName	lastName	email	pw
---------------	-----------	----------	-------	----

Order

<u>orderID</u>	<u>custID</u>	<u>artistID</u>	<u>artID</u>	orderDate
----------------	---------------	-----------------	--------------	-----------

Artist

<u>artistID</u>	firstName	lastName	email	bio
-----------------	-----------	----------	-------	-----

ArtWork

<u>artID</u>	<u>artistID</u>	title	descript	
--------------	-----------------	-------	----------	--

Mockup Table: (make sure ids match)

Order

orderID	customerID	artistID	artWorkID	orderDate	total
1111111	240111112	1000123	003		
1111112	240111112	1000124	001		
1111113	240111111	1000123	003		

No changes required in Customer or ArtWork tables.

Customer

customerID	firstName	lastName	eMail		
240111111	Ray				
240111112	Liz				

Artist

artistID	firstName	lastName	bio	eMail	
1000123	Sam				
1000124	Mary				

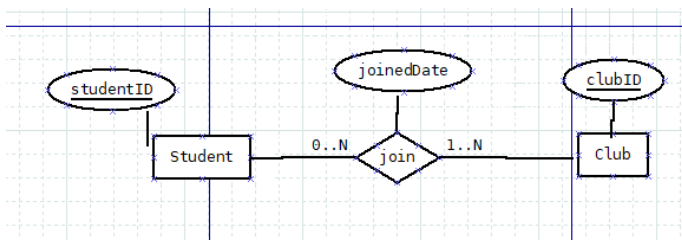
ArtWork

artistID	artWorkID	title	description	date	price
1000123	001				
1000123	003				
1000124	001				

Many-Many Relationships

There is no M-N relationship in the case study. We will use an example.

A student can join zero or many clubs and there may be zero or many students in a club.



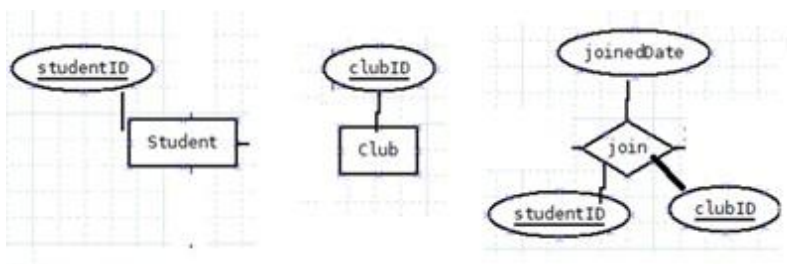
Need three tables, Student, Club, and Join.

Join links a particular student with a particular club. The associate attributes, if any, should be included in this table.

Student (studentID, firstName, ...)

Club(clubID, clubName,...)

Join(studentID, clubID, dateJoined, status,)



Mockup Tables:

Student

bannerID	firstName				
840111111	Ray				
840111112	Liz				

Club

clubID	name	da			
1231	ACM				
1232	MATH				

Student-Join-Club

clubID	studentID	dateJoined	due		
1231	840111111				
1231	840111112				
1232	840111111				

Back to our Case Study.

In logical design, model the relational tables and represent them in one of the forms below.

As discussed in the previous section, appropriate foreign keys (FK) are included in the entities to represent the relationships.

In logical design, relational tables can be represented in different format. Mockup tables are also created.

Logical Design (case study)

- **Represent tables in the following form (preferred method)**

- Artist(artistID, Name, Email, PasswordHash, Bio, address, ProfilePic)
- Artwork(artistID, ArtworkID, Title, Description, Price, ImageURL, Status)
 - artistID is an FK - links to Artist.artistID
- Customer(customerID, Name, Email, PasswordHash, address, shippingAddress)
- Orders (orderID, artistID, ArtworkID, CustomerID, OrderDate)
 - artistID is an FK- links to Artist.artistID
 - customerID is an FK- links to Customer.customerID
- OrderStatus (orderID, paidDate, payConfirmNumber,,paymentReceivedDate, shippedDate, shippingCarrier, trackingNumber,, receivedDate, archived)
 - orderID is an FK- linked to Orders.orderID

- **OR represent tables in text form**

Table: Artist

- - ArtistID (PK)
- - Name
- - Email
- - PasswordHash
- - Bio
- - ProfilePic

Table: Customer

- - CustomerID (PK)
- - Name
- - Email
- - PasswordHash
- - Address

Table: Artwork

- ArtistID (PK/FK)
- - ArtworkID (PK)
- - Title
- - Description
- - Price
- - ImageURL
- - Category
- - Status

Table: Orders

- - OrderID (PK)
- ArtistID (FK) → Artist.ArtistID
- - ArtworkID (FK) → Artwork.ArtworkID
- - CustomerID (FK) → Customer.CustomerID
- - OrderDate

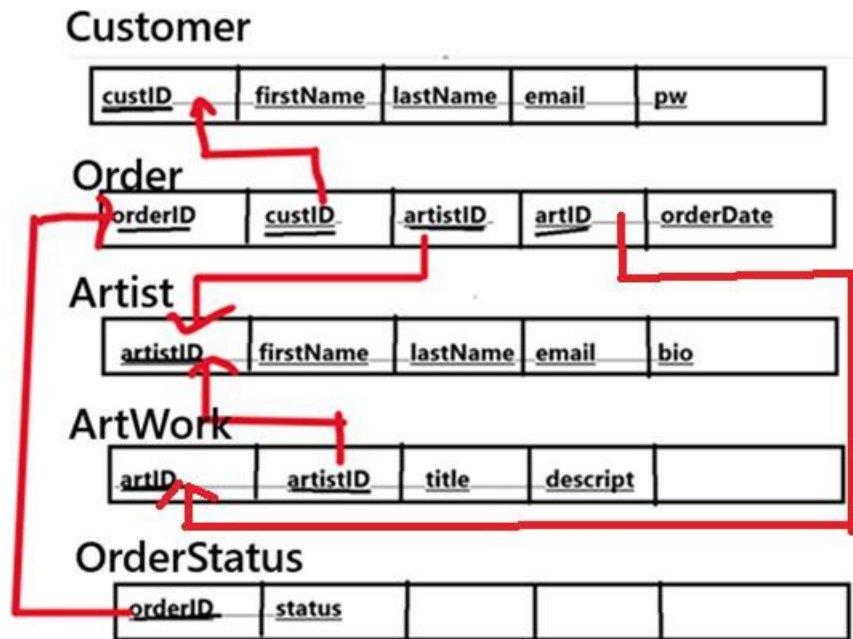
Table: OrderStatus

- - OrderID (PK/FK) → Order.OrderID
- status-
- PaidDate
- paymentMethod
- PaymentReceived
- - ShippingCarrier
- - TrackingNumber
- - ShippingDate

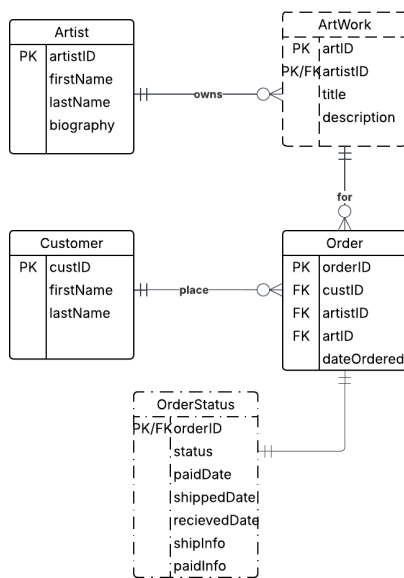
Table: Admin

- - AdminID (PK)
- - Name
- - Email
- - PasswordHash

- OR represent tables in the following form (ignore thin lines)



- OR Represent Tables as Physical Level E-R Diagram:

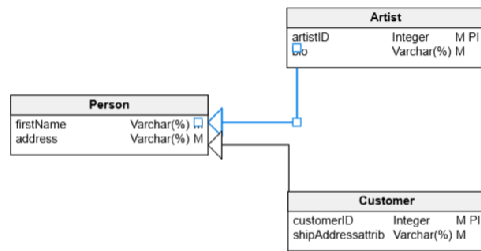


Summary: In physical design, create the relational tables and represent them in one of the forms above. In addition, create some mockup tables also.

Mapping EER Diagrams to Relational Schemas

Super- Sub/ General-Special/ Hierarchies – 3 methods

1. Create a table for the superset and one for each of the subsets Place primary key of the superset in each subset table
2. Create tables for each of the subsets, and no table for the superset Place all attributes of the superset in each of the subset tables
3. Create a single table with all the attributes of the superset and all the attributes of all subsets



Method-1 Create a table for the general (super) entity and each sub entities.

- Person(personID, Name, Email, PasswordHash, address,)

- Artist(personID, Bio, profilePicture)

- personID is PK links Person.personID

- Customer(personID, shippingAddress,)

- personID is PK links Person.personID

OR- if you want to keep separate formats for artistID and customerID

- Person(personID, Name, Email, PasswordHash, address,)

- Artist(artistID, personID, Bio, profilePicture)

- personID is PK links Person.personID

- Customer(customerID, personID, shippingAddress)

- personID is PK links Person.personID

Method-2: This is the original method.

- Artist(artistID, Name, Email, PasswordHash, Bio, address, ProfilePic)

- Customer(customerID, Name, Email, PasswordHash, address, shippingAddress)

This method not actually keeps the sup-super relationship. Used when splitting superclass into sub classes.

Method-3: Create a single table:

- Person(personID, role (artist or customer or both- since overlapping), Name, Email, PasswordHash, Bio, address, ProfilePicture,, address, shippingAddress)

This method is not good if the sub entities have many differentiating attributes. There will be many empty cells and storage will be wasted.

For this case study, we will use method-1.

The Final Relational Schema after mapping EERD

- Person(personID, Name, Email, PasswordHash, address,)

- Artist(personID, Bio, profilePicture)

- personID is PK links Person.personID

- Customer(personID, shippingAddress,)

- personID is PK links Person.personID

- Artwork(artistID, artworkID, Title, Description, Price, ImageURL, Status)

- artistID is an FK - links to Artist.personID

-- Orders (orderID, artistID, artworkID, customerID, OrderDate)

- artistID is an FK- links to Artist.personID
- customerID is an FK- links to Customer.personID

- OrderStatus (orderID, paidDate, payConfirmNumber,,paymentReceivedDate, shippedDate, shippingCarrier, trackingNumber, receivedDate, archived)

- orderID is an FK- linked to Orders.orderID

Assumption:

A Person may be an Artist, Customer, or Both.

When registering first time, they have the option to select Artist, Customer, or Both.

Even after registering, they may add a role- both dashboards will include an option to add the other role.

Step 4: Database Implementation (create Database & test)

Database: MySQL or PostgreSQL (check PostgreSQL help document in Canvas)

Use SQL commands to create tables, specify primary keys, foreign keys (manage relationships) , and other constraints such as NOT NULL, UNIQUE, etc., .

OR we can also use GUI to create the DB schema.

Creating Server, Database, and Tables

Run PostgreSQL; select a Server or create a Server and select.

Click on Databases and create ArtGallery1.

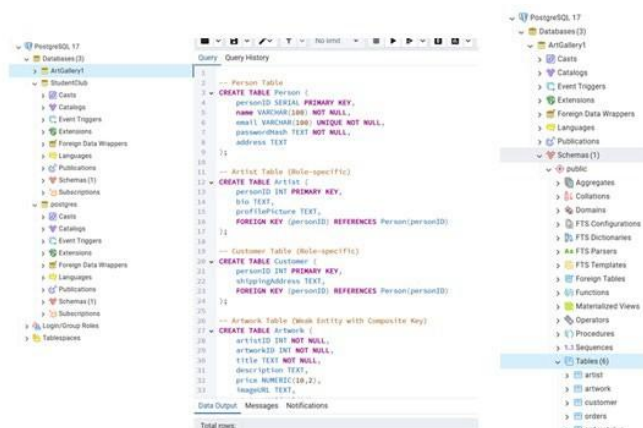
Click-on ArtGallery1, and from Tools select Query tool.

Write your SQL code to create tables (given below).

Execute the statements (F5). It would confirm that your queries ran successfully.

You can see the tables – under schemas-tables.

Note that, -- is used for comments in PostgreSQL



-- Person table

```
CREATE TABLE Person (
  personID SERIAL PRIMARY KEY,
  firstName VARCHAR(100) NOT NULL,
  lastName VARCHAR(100) NOT NULL,
```

```
email VARCHAR(100) UNIQUE NOT NULL,
passwordHash TEXT NOT NULL,
address TEXT
);
--Artist table
CREATE TABLE Artist (
    personID INT PRIMARY KEY,
    bio TEXT,
    profilePicture TEXT,
    FOREIGN KEY (personID) REFERENCES Person(personID)
);
// Customer table
CREATE TABLE Customer (
    personID INT PRIMARY KEY,
    shippingAddress TEXT,
    FOREIGN KEY (personID) REFERENCES Person(personID)
);
-- Artwork table (Weak entity: composite PK)
CREATE TABLE Artwork (
    artistID INT NOT NULL,
    artworkID INT NOT NULL,
    title TEXT NOT NULL,
    description TEXT,
    price NUMERIC(10,2),
    imageURL TEXT,
    status VARCHAR(20),
    PRIMARY KEY (artistID, artworkID),
    FOREIGN KEY (artistID) REFERENCES Artist(personID)
```

```

);

--Orders table

CREATE TABLE Orders (
    orderID SERIAL PRIMARY KEY,
    artistID INT NOT NULL,
    artworkID INT NOT NULL,
    customerID INT NOT NULL,
    orderDate DATE NOT NULL,
    FOREIGN KEY (artistID, artworkID) REFERENCES Artwork(artistID, artworkID),
    FOREIGN KEY (customerID) REFERENCES Customer(personID)
);

--OrderStatus table

CREATE TABLE OrderStatus (
    orderID INT PRIMARY KEY,
    paidDate DATE,
    payConfirmNumber TEXT,
    paymentReceivedDate DATE,
    shippedDate DATE,
    shippingCarrier TEXT,
    trackingNumber TEXT,
    receivedDate DATE,
    archived BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (orderID) REFERENCES Orders(orderID)
);

```

Run PostgreSQL; select a Server, click on Databases and create ArtGallery1.

Click-on ArtGallery1, and from Tools select Query tool.

Write your SQL code to create tables. It would confirm that your queries ran successfully.

You can see the tables – under schemas-tables.

Screenshots of the table structures. (right-click on a table, select properties, select columns)

The screenshot shows the 'person' table structure. The 'Columns' tab is selected. The table has six columns: personid, firstname, lastname, email, passwordhash, and address. The 'personid' column is the primary key.

Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
personid	integer			☒	☒	nextval('personid_seq')
firstname	character varying	100		☒	☐	
lastname	character varying	100		☒	☐	
email	character varying	100		☒	☐	
passwordhash	text			☒	☐	
address	text			☐	☐	

The screenshot shows the 'artist' table structure. The 'Columns' tab is selected. The table has four columns: personid, bio, profilepicture, and artworkid. The 'personid' column is the primary key.

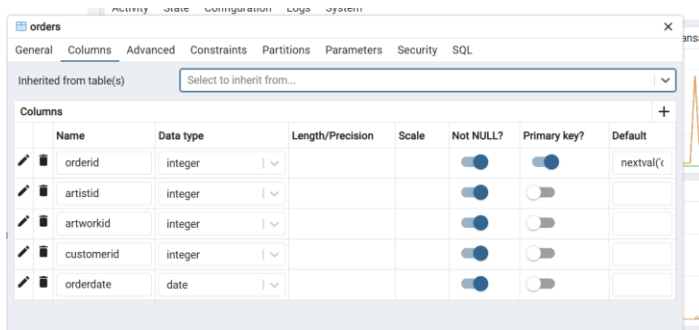
Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
personid	integer			☒	☒	
bio	text			☐	☐	
profilepicture	text			☐	☐	
artworkid	integer			☐	☐	

The screenshot shows the 'customer' table structure. The 'Columns' tab is selected. The table has two columns: personid and shippingaddress. The 'personid' column is the primary key.

Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
personid	integer			☒	☒	
shippingaddress	text			☐	☐	

The screenshot shows the 'artwork' table structure. The 'Columns' tab is selected. The table has eight columns: artistid, artworkid, title, description, price, imageurl, and status. The 'artistid' and 'artworkid' columns are primary keys.

Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
artistid	integer			☒	☒	
artworkid	integer			☒	☒	
title	text			☒	☐	
description	text			☐	☐	
price	numeric	10	2	☐	☐	
imageurl	text			☐	☐	
status	character varying	20		☐	☐	



Inserting Some Mock-up Data

Click-on ArtGallery1, and from Tools select Query tool.

Write your SQL code to add some mock-up rows to your tables (given below).

Make sure about the constraints. For example, FKs match with PKs and others. Otherwise, DB system will complain.

Execute the statements (F5). It would confirm that your queries ran successfully.

--Person – 5 are Artists; 5 are Customers; 2 of them are both Artist and Customer

INSERT INTO Person (firstName, lastName, email, passwordHash, address)

VALUES

```
( 'Alice', 'Monroe', 'alice@example.com', 'hash1', '123 Art St, NY'),      -- 1
( 'Ben', 'Carter', 'ben@example.com', 'hash2', '456 Canvas Ave, CA'),    -- 2
( 'Clara', 'Young', 'clara@example.com', 'hash3', '789 Studio Ln, TX'),  -- 3
( 'David', 'Smith', 'david@example.com', 'hash4', '12 Market St, NC'),  -- 4
( 'Eva', 'Green', 'eva@example.com', 'hash5', '34 River Rd, SC'),       -- 5
( 'John', 'Doe', 'john@example.com', 'hash6', '67 Sunset Blvd, FL'),    -- 6
( 'Jane', 'Roe', 'jane@example.com', 'hash7', '89 River Pkwy, GA'),     -- 7
( 'Mike', 'Brown', 'mike@example.com', 'hash8', '21 Art Dr, TX'),       -- 8
( 'Linda', 'White', 'linda@example.com', 'hash9', '43 Collector Ave, AL'), -- 9
( 'Sam', 'Lee', 'sam@example.com', 'hash10', '99 Future Rd, WA');       -- 10
```

--Artist – IDs 1, 2, 3, 4, 5

INSERT INTO Artist (personID, bio, profilePicture)

VALUES

(1, 'Landscape and portrait artist.', 'alice.jpg'),

(2, 'Abstract painter.', 'ben.jpg'),

(3, 'Mixed media urban artist.', 'clara.jpg'),

(4, 'Oil painting specialist.', 'david.jpg'),

(5, 'Watercolor nature scenes.', 'eva.jpg');

--Customer – IDs 1, 3, 6, 7, 8, 9

INSERT INTO Customer (personID, shippingAddress)

VALUES

(1, '123 Art St, NY'),

(3, '789 Studio Ln, TX'),

(6, '67 Sunset Blvd, FL'),

(7, '89 River Pkwy, GA'),

(8, '21 Art Dr, TX'),

(9, '43 Collector Ave, AL');

--Artwork – 5 total (from Artists 1–5)

INSERT INTO Artwork (artistID, artworkID, title, description, price, imageURL, status)

VALUES

(1, 1, 'Sunset Bliss', 'Acrylic sunset painting', 500.00, 'sunset.jpg', 'available'),

(2, 1, 'Forest Whisper', 'Oil forest trail', 750.00, 'forest.jpg', 'available'),

(3, 1, 'Urban Life', 'Mixed media cityscape', 900.00, 'urban.jpg', 'available'),

(4, 1, 'Tranquil Morning', 'Serene lake in watercolor', 350.00, 'lake.jpg', 'available'),

```
(5, 1, 'Bloom Burst', 'Colorful floral watercolor', 400.00, 'floral.jpg', 'available');
```

```
--Orders – 5 total (from the 6 customers)
```

```
INSERT INTO Orders (artistID, artworkID, customerID, orderDate)
```

```
VALUES
```

```
(2, 1, 1, '2025-06-01'), -- Alice buys from Ben
```

```
(3, 1, 3, '2025-06-02'), -- Clara buys from Clara (intentional- to test)
```

```
(1, 1, 6, '2025-06-03'), -- John buys from Alice
```

```
(4, 1, 7, '2025-06-04'), -- Jane buys from David
```

```
(5, 1, 8, '2025-06-05'); -- Mike buys from Eva
```

```
--OrderStatus
```

```
INSERT INTO OrderStatus (orderID, paidDate, payConfirmNumber, paymentReceivedDate,  
shippedDate, shippingCarrier, trackingNumber, receivedDate, archived)
```

```
VALUES
```

```
(1, '2025-06-01', 'ZL101', '2025-06-02', '2025-06-03', 'UPS', 'UPS101', '2025-06-05', TRUE),
```

```
(2, '2025-06-02', 'ZL102', '2025-06-03', NULL, NULL, NULL, NULL, FALSE),
```

```
(3, '2025-06-03', 'ZL103', '2025-06-04', '2025-06-05', 'FedEx', 'FDX103', '2025-06-07', TRUE),
```

```
(4, '2025-06-04', 'ZL104', '2025-06-05', '2025-06-06', 'UPS', 'UPS104', NULL, FALSE),
```

```
(5, '2025-06-05', 'ZL105', '2025-06-06', '2025-06-07', 'DHL', 'DHL105', '2025-06-09', TRUE);
```

Screenshots of the Table with rows (right-click on the table, select view/edit)

Count Rows
Create
Delete
Delete (Cascade)
Refresh...
Restore...
Backup...
Import/Export Data...
Reset Statistics
ERD For Table
Maintenance...
Scripts
Truncate
View/Edit Data
Search Objects...

	personid [PK] integer	firstname character varying (100)	lastname character varying (100)	email character varying (100)	passwordhash text	address text
1	1	Alice	Monroe	alice@example.com	hash1	123 Art St, NY
2	2	Ben	Carter	ben@example.com	hash2	456 Canvas Ave, CA
3	3	Clara	Young	clara@example.com	hash3	789 Studio Ln, TX
4	4	David	Smith	david@example.com	hash4	12 Market St, NC
5	5	Eva	Green	eva@example.com	hash5	34 River Rd, SC
6	6	John	Doe	john@example.com	hash6	67 Sunset Blvd, FL
7	7	Jane	Roe	jane@example.com	hash7	89 River Pkwy, GA
8	8	Mike	Brown	mike@example.com	hash8	21 Art Dr, TX
9	9	Linda	White	linda@example.com	hash9	43 Collector Ave, AL
10	10	Sam	Lee	sam@example.com	hash10	99 Future Rd, WA

Similarly include screenshots of other tables also.

Step 5: Design Forms and Reports (interface design)

(need not to be perfect- you can just sketch it or use online tools- wireframes)

Forms:

- Login (all)

Login

Email

Password

Login as:

Customer

Login

New user? Register as
[Customer](#) | [Artist](#)

- Artist registration

Artist Registration

Name:

Email:

Password:

Biography:

Register

- Artwork upload form (Artist already logged-in)

Upload Artwork

Title:

Description:

Category:

Price (USD):

Upload Image:

Choose File

No file chosen

Upload Artwork

Customer registration

Customer Registration

Name:

Email:

Password:

Shipping Address:

Register

- Order Form

Place Your Order

Your Name:

John Doe

Email:

john@example.com

Select Artist:

-- Select Artist --

Select Artwork:

-- Select Artwork --

Shipping Address (change if different):

456 Oak Street, Springfield, IL 62704

Payment Method:

Zelle

Payment Confirmation Number:

Enter Zelle reference number

Submit Order

Unified Order Management View form, accessible by customers, artists, and admins, and will be different based on roles and responsibilities.

It displays order status and allows status updates based on user roles and offers a comprehensive order lifecycle view, including:

Ordered (by customer): Paid (by customer): Payment Received (by artist)

Shipped (by artist): Item Received (by customer)

Later: Cancelled / Returned / Return Received / Reimbursed / Reimbursement Received

Order Summary

Order ID:
Artwork:
Customer:
Order Date:

Ordered:
Paid:
Payment Received:
Shipped:
Item Received:
Archived:

Order **Status:**

Customer: Payment Submission

Payment Zelle Confirmation Notes:

E.g., Zelle sent on [date], last 4 digits...

[Submit Payment Info](#)

Artist: Confirm Payment & Shipping

☐ Confirm Zelle Payment Received

Shipping Carrier:

Tracking Number:

[Update Shipping Info](#)

Customer: Confirm Receipt

☐ I have received the artwork.

[Confirm Receipt](#)

Admin Controls

☐ Cancel Order
☐ Archive Order

[Submit Admin Action](#)

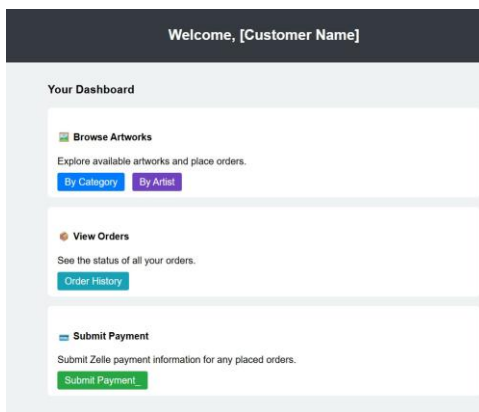
Admin can see all orders, but others can see only their orders.

Order Management

Order ID	Date	Customer	Artist	Artwork	Status
ORD1023	2025-05-28	Alice Johnson	Artist A	Sunset Horizon	Paid
ORD1024	2025-05-27	Bob Smith	Artist B	Ocean Dreams	Shipped
ORD1025	2025-05-26	Carol Lee	Artist C	City Lights	Ordered

Output / Reports:

-Customer Dashboard



- Artist Dashboard

-Admin Dashboard

All Artworks- By Category

All Artworks- By Artist

Order history for customers

Sold and available artworks for artists

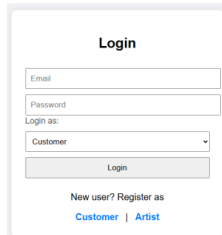
Order history for Admin

Archived transactions for admin

Step 6: Query Design and Testing (Database Implementation)

Forms:

- Login (all)



Assumption: A person can login as one of the two roles they registered for- not both (if they registered for both)

Request to DB:

Given a username (email) and password, check if a person exists

Then, check if that person is registered as the selected role (either Artist or Customer)

If not in that role, return an error

We'll assume plaintext password comparison for now (you'd use hashing in production).

Scenario-1: Assume a user gives email, password, and select their role

SQL Code:

```
SELECT personID
```

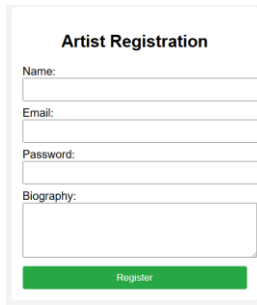
```
FROM person
```

```
WHERE email = 'alice@example.com'
```

```
AND passwordhash = 'hash1';
```

- If this returns **no rows**, return Error: Invalid email or password.
- If it returns a row, note the personid.

- Artist registration



Artist Registration

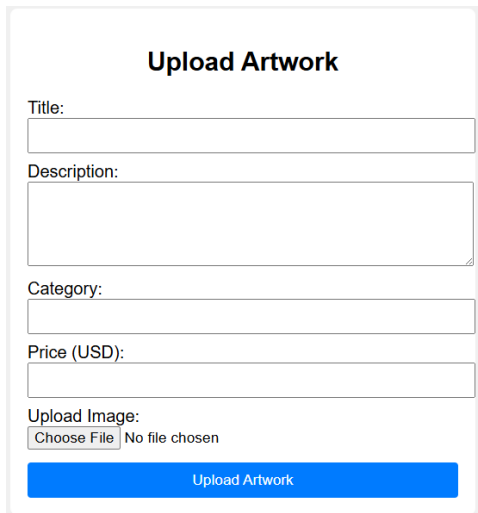
Name:

Email:

Password:

Biography:

- Artwork upload form (Artist already logged-in)



Upload Artwork

Title:

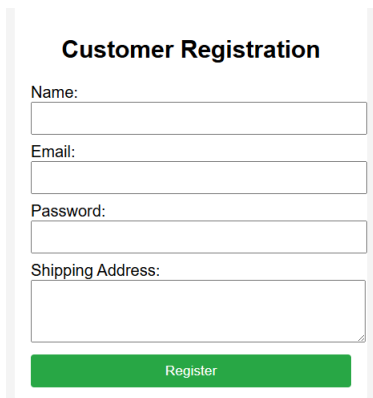
Description:

Category:

Price (USD):

Upload Image:
 No file chosen

- Customer registration



Customer Registration

Name:

Email:

Password:

Shipping Address:

- Order Form

Place Your Order

Your Name:

John Doe

Email:

john@example.com

Select Artist:

-- Select Artist --

Select Artwork:

-- Select Artwork --

Shipping Address (change if different):

456 Oak Street, Springfield, IL 62704

Payment Method:

Zelle

Payment Confirmation Number:

Enter Zelle reference number

Submit Order

Unified Order Management View form, accessible by customers, artists, and admins, and will be different based on roles and responsibilities.

It displays order status and allows status updates based on user roles and offers a comprehensive order lifecycle view, including:

Ordered (by customer): Paid (by customer): Payment Received (by artist)

Shipped (by artist): Item Received (by customer)

Later: Cancelled / Returned/ Return Received / Reimbursed / Reimbursement Received

Step 7: Program Implementation

The database work in Step 4 was built and tested in PostgreSQL — the schema, constraints, and mock data above all use PostgreSQL syntax (SERIAL, etc.) and were verified through pgAdmin. Step 7 should build on that choice rather than switching database engines, so the technology stack below is selected to keep PostgreSQL and to bring in only as much web-development tooling as is needed to demonstrate the schema through a working interface.

Recommended Stack

- Frontend: HTML5, CSS3, JavaScript, Bootstrap (renders the Step 5 wireframes directly)
- Backend: Python Flask, using Jinja2 templates and the psycopg2 driver
- Database: PostgreSQL (unchanged from Step 4)

Features to Implement

- Secure login for artists and customers
- Dashboard views based on user role
- Artwork display and order tracking
- Manual Zelle payment confirmation process
- Shipping update by the artist and receipt confirmation by the customer
- Auto-archival of completed orders

Comparing Stacks for a DBMS Course Demo

The database itself is already finished (Step 4), so the remaining choice is which web layer is fastest and clearest for demonstrating it, without the web framework distracting from the database design that is the actual point of this project. The comparison below evaluates the options against that goal.

Stack	Setup effort	Learning curve	Keeps existing PostgreSQL work?	Fit for this DBMS project
XAMPP (PHP + MySQL/MariaDB)	Lowest — single installer bundles Apache, MySQL, PHP, and phpMyAdmin	Very low — PHP embeds directly in HTML, no build tools or virtual environments	No — would require redoing the Step 4 schema in MySQL instead of PostgreSQL	Fastest path to a working demo and abundant tutorials for this exact type of CRUD app, but PHP mixed with HTML gets messy past a few pages, and it means abandoning the PostgreSQL work already done
Flask (Python) + PostgreSQL — RECOMMENDED	Low-medium — pip install flask and psycopg2-binary inside one virtual environment	Low, especially if Python is already familiar	Yes — psycopg2 runs the Step 4/6 SQL almost unchanged	Best fit: keeps the hand-designed schema and constraints front and center, Jinja2 templates plus Bootstrap render the Step 5 wireframes directly, and the project stays focused on the

Stack	Setup effort	Learning curve	Keeps existing PostgreSQL work?	Fit for this DBMS project
				database rather than the web framework
Node.js + Express	Medium — npm ecosystem, pg module, plus a template engine (EJS) or separate frontend	Medium-high — asynchronous patterns and more moving parts	Yes — pg module can reuse the existing schema	Workable if JavaScript is already familiar, but for a DBMS-focused demo it adds tooling complexity that doesn't add insight into the database design itself
Django + PostgreSQL	Medium — project/app scaffolding, migrations	Medium — Django's ORM is its own layer to learn	Partial — Django's ORM tends to auto-generate schema, which can bury the hand-crafted PK/FK relationships	Not ideal for this purpose: the ORM abstracts away the raw SQL and constraints that are the actual point of the exercise

Recommendation: Flask + PostgreSQL + Jinja2 + Bootstrap. It adds the least new tooling on top of the PostgreSQL schema already built in Step 4, keeps the hand-written SQL and JOINS from Steps 2–3 visible in the application code rather than hidden behind an ORM, and maps directly onto the Step 5 wireframes. If the course specifically requires MySQL, or the absolute lowest setup effort is the priority and PHP is acceptable, XAMPP is the other reasonable choice — but that path means rebuilding Step 4's schema in MySQL rather than reusing the PostgreSQL work already done.

Implementation Plan

- Create an implementation plan.
- Create test cases.
- Implement and test — integrate incrementally.

Step 8: Additional Features:

Cancellation and Return Policy

Order Cancellation

- Customers may request order cancellation **within 24 hours** of placing the order, provided the artist has not yet shipped the artwork.
- To cancel an order, customers must contact support with their **Order ID** and reason for cancellation.
- If the artist has already shipped the artwork, the order **cannot be canceled**.

Returns and Refunds

- Due to the unique and fragile nature of artwork, **returns are generally not accepted.**
- Exceptions will be made only if:
 - The artwork arrives **damaged**, or
 - The item received is **significantly different** from the listing.
- Customers must report issues **within 3 days** of receiving the item, with **photos of the artwork and packaging.**
- Upon approval, the customer may return the item. Refunds will be processed **after the artist receives the returned artwork** in its original condition.

Shipping for Returns

- Return shipping is the responsibility of the customer unless the return is due to an error by the artist (e.g., wrong item).
- The return must be shipped using a **trackable shipping service.**

Refund Timeline

- Approved refunds will be processed within **7–10 business days** after the returned item is received and inspected.